

gemstone-rs Shared Core Integration

How gemstone-py-native can wrap the Rust core



Generated from the Markdown sources in gemstone-rs/docs.

Contents

Shared Core Integration

What Should Move Into Rust

What Should Stay Python-Specific

PyO3 Wrapper Shape

Migration Plan

Shared Core Integration

The long-term direction is:

```
gemstone-gci
  Low-level dynamic libgci loader and raw GCI calls.

gemstone-rs
  Safe Rust API: Config, Session, Oop, Value, browser, codegen, BridgeRoot.

gemstone-py-native
  Thin PyO3 wrapper around the Rust core.

gemstone-py
  Python API, async/web adapters, examples, docs, and compatibility layer.
```

This keeps Python and Rust from growing separate native bridges.

What Should Move Into Rust

- dynamic GCI library loading
- OOP constants and conversions
- session login/logout
- eval, execute, perform
- global get/put
- string, symbol, array, and dictionary marshalling
- transaction commit/abort
- browser primitives
- codegen discovery primitives

What Should Stay Python-Specific

- Pythonic `Session` and async wrappers
- FastAPI, Litestar, Django examples
- Python package extras such as `gemstone-py[fast]`
- backward-compatible return behavior
- Python docs and notebooks

PyO3 Wrapper Shape

`gemstone-py-native` should become a small adapter:

```
Python call -> PyO3 wrapper -> gemstone-rs Session -> gemstone-gci -> libgcirpc
```

The wrapper should expose stable operations first:

- `login(config)`
- `eval(source)`
- `execute(source)`
- `perform(oop, selector, args)`
- `commit()`
- `abort()`
- `logout()`

Only after that should it expose higher-level browser, codegen, and BridgeRoot operations.

Migration Plan

1. Keep `gemstone-rs` independent and publishable.
2. Add a small `PyO3` crate that depends on `gemstone-rs`.
3. Replace duplicated native loading code in `gemstone-py-native`.
4. Run the existing `gemstone-py` live tests through the Rust-backed native path.
5. Keep pure Python fallback behavior available.

The main design rule: Rust owns the native bridge; Python owns Python ergonomics.

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.